ISC4 Mini Project

Biyao WANG
Saad Bin Asad
Responsable, Enseignant: Bonnifait Philippe

January 2025

Contents

1 Start-up decision and calculation of the acceleration **3**

1.1 Calculation of a_f and T_f . . . 3

1.1.1 State Vector and Target Conditions . . . 3

1.1.2 Calculation of T_f . . . 3

1.1.3 Calculation of a_f . . . 3

1.1.4 Results . . . 4

1.2 Determination of vehicle pair [i,i+1] and of t_0 . . . 4

1.2.1 First Condition . . . 4

1.2.2 Second Condition . . . 5

1.3 Simulation . . . 5

1.3.1 Script for the Open-loop control strategy . . . 5

1.3.2 Results and Graphs . . . 6

2 Taking sensor measurement uncertainties into account **11**

3 Closed-loop control **12**

3.1 Closed-loop feedback equations . . . 12

3.2 Calculation of closed-loop parameters . . . 13

3.3 Simulation . . . 13

3.3.1 Script for the Closed-loop control strategy . . . 13

3.3.2 Results and Graphs . . . 14

3.3.3 Impact of Negative D_{xx}: . . . 15

3.3.4 Evaluation of performance of the closed-loop strategy: . . . 16

4 Additional Work **18**

1 Start-up decision and calculation of the acceleration

1.1 Calculation of a_f and T_f

1.1.1 State Vector and Target Conditions

Initial and Target States

- Initial state:

$$X(t_0) = \begin{bmatrix} -L_0 \\ 0 \end{bmatrix}$$

This indicates that the vehicle's initial position is $-L_0$ and its initial velocity is 0.

- Target state:

$$X(t_0 + T_f) = \begin{bmatrix} -\frac{3L_0}{4} \\ v_f \end{bmatrix}$$

This indicates that the vehicle's target position is $-\frac{3L_0}{4}$ and its target velocity is v_f.

Equations of Uniformly Accelerated Motion

The basic equations for uniformly accelerated motion are:

1. Position equation:

$$x(t) = x(t_0) + \dot{x}(t_0) \cdot t + \frac{1}{2} a_f t^2$$

2. Velocity equation:

$$\dot{x}(t) = \dot{x}(t_0) + a_f t$$

1.1.2 Calculation of T_f

From the velocity equation:

$$\dot{x}(t_0 + T_f) = v_f$$

Since the initial velocity is 0:

$$v_f = 0 + a_f T_f$$

Solving for T_f:

$$T_f = \frac{v_f}{a_f}$$

1.1.3 Calculation of a_f

From the position equation, when $t = T_f$:

$$x(t_0 + T_f) = -\frac{3L_0}{4}$$

Substituting into the position equation:

$$-\frac{3L_0}{4} = -L_0 + 0 \cdot T_f + \frac{1}{2} a_f T_f^2$$

Simplifying:

$$-\frac{3L_0}{4} = -L_0 + \frac{1}{2} a_f T_f^2$$

Rearranging terms:

$$L_0 - \frac{3L_0}{4} = \frac{1}{2} a_f T_f^2$$

$$\frac{L_0}{4} = \frac{1}{2} a_f T_f^2$$

Substituting $T_f = \frac{v_f}{a_f}$:

$$\frac{L_0}{4} = \frac{1}{2} a_f \left(\frac{v_f}{a_f}\right)^2$$

Simplifying:

$$\frac{L_0}{4} = \frac{1}{2} \cdot \frac{v_f^2}{a_f}$$

Solving for a_f:

$$a_f = \frac{2v_f^2}{L_0}$$

1.1.4 Results

- Constant acceleration:

$$a_f = \frac{2v_f^2}{L_0}$$

- Required time:

$$T_f = \frac{L_0}{2v_f}$$

1.2 Determination of vehicle pair [i,i+1] and of t_0

1.2.1 First Condition

To ensure safe insertion, there must be sufficient space between vehicles i and $i+1$. The required spacing consists of the following components:

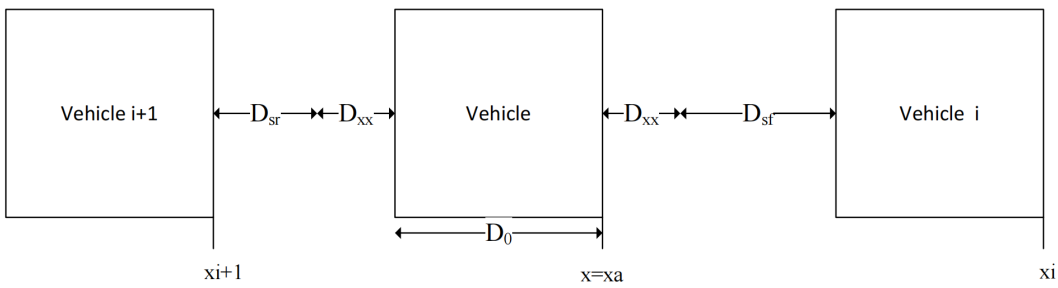


Figure 1: Insertion with safety margin

1. **Front safety distance** D_{sf}: This is the minimum distance required to avoid collision with the vehicle ahead, defined as:

$$D_{sf} = D_0 + \frac{v_f}{2}$$

2. **Rear safety distance** D_{sr}: This is the minimum distance required to avoid being rear-ended by the following vehicle, defined as:

$$D_{sr} = D_0 + \frac{v_f}{4}$$

3. **Vehicle length** D_i: The physical length of the vehicle ahead must be considered as part of the insertion gap.

4. **Additional safety margin** D_{xx}: An extra safety buffer to ensure secure insertion.

The total spacing required for a successful insertion can be expressed as:

$$x_i(t) - x_{i+1}(t) > D_{sf} + D_0 + D_{sr} + D_i$$

In practice, there is also a distance margin D_{xx}.
So, the first condition will be(as was shown in the picture):

$$x_i(t) - x_{i+1}(t) > D_{sf} + D_0 + D_{sr} + D_i + 2D_{xx}$$

1.2.2 Second Condition

1. Formula for ideal position for the insertion point x_a

From the problem setup, the position of the insertion point is given as:

$$x_a(t) = x_{i+1}(t) + D_{sr} + D_0 + D_{xx}$$

Where:

- $x_{i+1}(t)$: Position of vehicle $i+1$ at time t.
- $D_{sr} = D_0 + \frac{v_f}{4}$: Rear safety distance.
- D_0: Length of the vehicle.
- D_{xx}: Additional safety margin.

2. Time T_a for the Insertion Point to Reach $-\frac{3L_0}{4}$

The insertion point x_a will move at a constant speed v_f to reach $-\frac{3L_0}{4}$. Thus, we have:

$$x_a(t + T_a) = -\frac{3L_0}{4}$$

$$x_a(t) = x_{i+1}(t) + D_{sr} + D_0 + D_{xx}$$

$$T_a = \frac{-\frac{3L_0}{4} - x_a(t)}{v_f}$$

$$T_a = \frac{-\frac{3L_0}{4} - (x_{i+1}(t) + D_{sr} + D_0 + D_{xx})}{v_f}$$

If $T_a > T_f$, the vehicle has enough time to reach the insertion point.

If $T_a \leq T_f$, the current vehicle pair does not satisfy the insertion condition, and another vehicle pair should be considered.

1.3 Simulation

1.3.1 Script for the Open-loop control strategy

We create a new script for the open-loop acceleration. The following changes were made to the script, using the calculations performed previously:

```
% Calculate required constants for control
vf_est = vf;
Tf = L0 / (2 * vf_est); % Time to reach -3L0/4 at constant acceleration
af = 2 * vf_est^2 / L0; % Constant acceleration value
```

```
if (N_file > 1) % Frequent case with several cars in the line
    for i=1:N_file-1
            % Check inter-vehicle distance
            Deltax = File(i).pos_est - File(i+1).pos_est - File(i).long;% Inter-vehicle distance
            if Deltax >(D0 + Dsar + Dsav)    % If there's enough room, we wonder if we can get there in time
                % Time it will take for the insertion point to reach the abscissa -3*L0/4
                Dxx = (Deltax - ( D0 + Dsar + Dsav ))/2;
                x_a = File(i+1).pos_est + Dsar + D0 + Dxx;
                Ta = (- 3*L0/4 - x_a)/vf_est; % Time for insertion point
                if Ta>Tf   % We have enough time to get there
                    t0=t+Ta-Tf; % Calculation of the starting time
                    acc_B0 = af; % The acceleration is calculated
                    demar=1;
                    N_suivi=i;% Select this vehicle pair
                    disp(['We insert between vehicles',Couleurs(File(N_suivi).icouleur,:),' and ',Couleurs(File(N_suivi+1).icouleur,:)]);
                    disp(['Distance between vehicles i et i+1 = ',num2str(File(i).pos_est -File(i+1).pos_est -File(i).long)]);
                    break; % Exit the loop because the solution has been found
                end
            end
        end
    end
```

In $simufile.m$ we set $Display_fig = 0$ and $deltaT = 0.01$ to speed up the simulations and reduce calculation errors. In order to test it with a very high traffic flow, we set $debit = 5000$; in $parameters.m$.

1.3.2 Results and Graphs

Acceleration Example

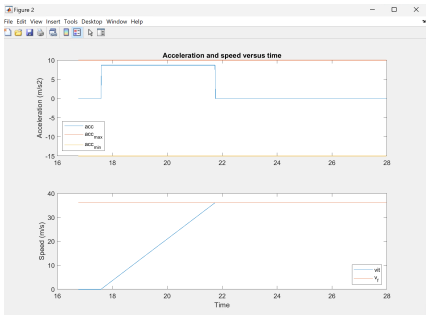
We can test the Acceleration Example by launching the main script $simufile.m$ and setting $question = 0$ in the script.

```
case 0 % program example
    % the estimated information used for insertion is
    % "pos_est" and "vf_est".
    % Here they are not noisy
    for i= 1:N_file, File(i).pos_est = File(i).pos;end
    vf_est = vf;
    acceleration_example; % Script execution
```

Figure 2: question=0

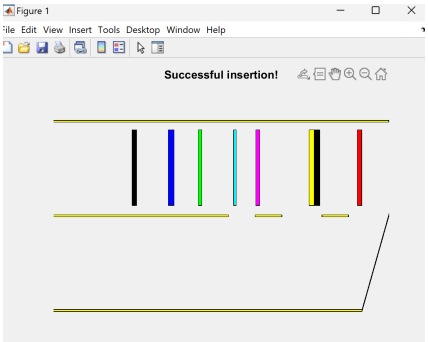
Following are some of the Simulation Window and the velocity and acceleration curves:

(a) Simulation Window

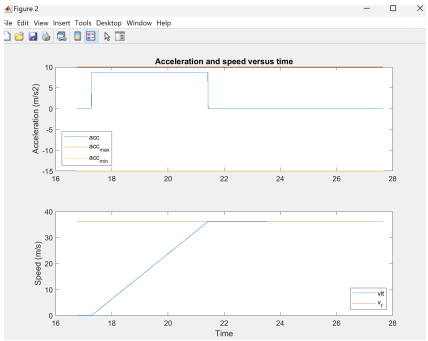


(b) Acceleration and speed versus time

Figure 3: Acceleration Example - Unsafe insertion

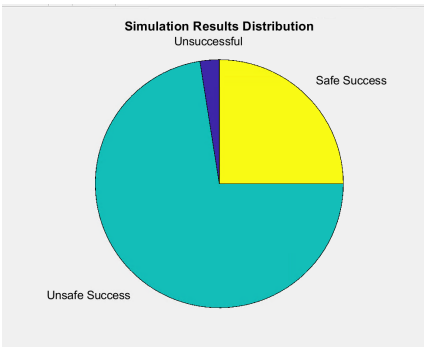


(a) Simulation Window

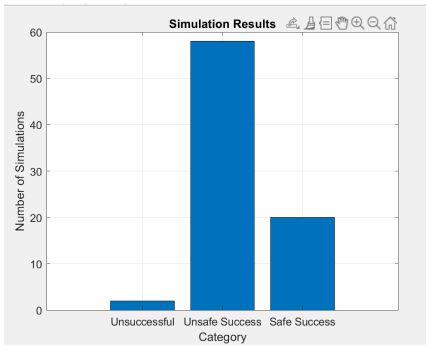


(b) Acceleration and speed versus time

Figure 4: Acceleration Example - Safe insertion



(a) Acceleration example (pie-chart)



(b) Acceleration example (bar-chart)

Figure 5: Acceleration Example - Analysis of Diary

```
Unsuccessful insertions: 2
Successful but unsafe insertions: 58
Successful and safe insertions: 20
```

Figure 6: Result of Acceleration Example

Results

Insertion Success and Failure: A total of **80 simulations** were performed, achieving an overall **success rate of 97.5%** (78 successful insertions), but only **25% of the insertions were safe**.

- Unsuccessful: **2.5%**.
- Unsafe Success: **72.5%**.
- Safe Success: **25%**.

Comments

1. High Proportion of Unsafe Success:
   - Unsafe success (72.5%) significantly outweighs safe success (25%), indicating that the primary focus of the program is achieving insertion, with insufficient consideration for safety.
   - The program does not handle safety distance constraints or acceleration saturation, leading to aggressive insertion behavior.
2. Low Safe Insertion Rate:
   - The safe insertion rate is only 25%, suggesting that the `acceleration_example` program oversimplifies the modeling of vehicle dynamics and the insertion environment.
   - Limitations include:
     - Lack of comprehensive consideration for neighboring vehicle behaviors.
3. Low Failure Rate:
   - The failure rate is only 2.5%, indicating good capability in finding insertion opportunities. However, this high success rate comes at the cost of safety, with potential risks of collisions.
4. Cause Analysis :
   - `acceleration_example` relies only on position and velocity estimates without fully considering safety distances or vehicle dynamics.
   - The simulation does not incorporate sensor noise, which could introduce significant safety risks in real-world scenarios.
   - The program likely uses short insertion times and high accelerations, leading to unsafe but successful insertions.

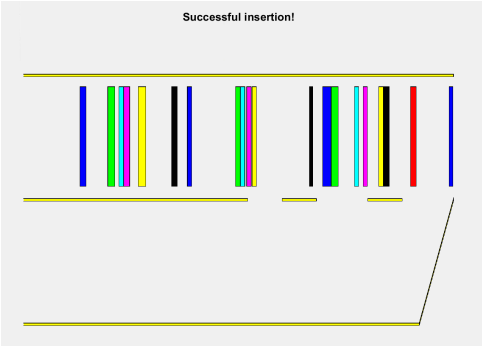
Open-loop control strategy

We can test the Acceleration Example by launching the main script $simufile.m$ and setting $question = 1$ in the script.

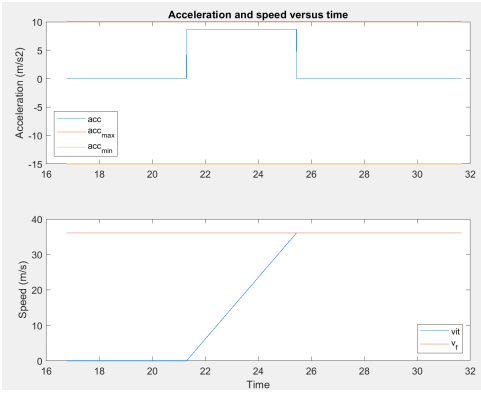
```
case 1 % question 1
    % estimated information is not noisy
    for i= 1:N_file, File(i).pos_est = File(i).pos;end
    vf_est = vf;
    acceleration_open_loop; % Script execution
```

Figure 7: question=1

Following are some of the Simulation Window and the velocity and acceleration curves:

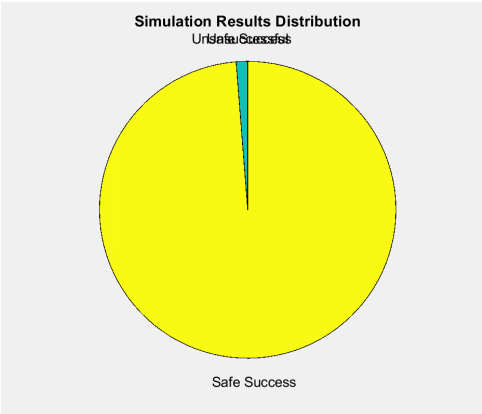


(a) Simulation Window

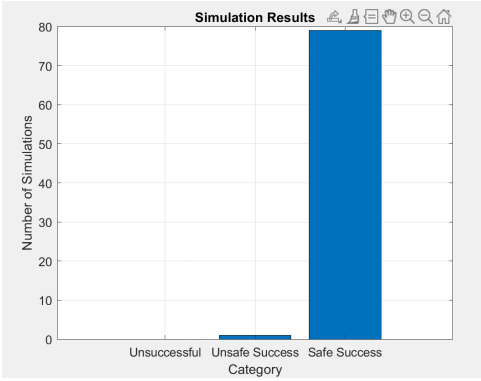


(b) Acceleration and speed versus time

Figure 8: Open loop - Safe insertion



(a) Open loop (pie-chart)



(b) Open loop (bar-chart)

Figure 9: Open loop - Analysis of Diary

```
Unsuccessful insertions: 0
Successful but unsafe insertions: 1
Successful and safe insertions: 79
```

Figure 10: Result of open loop

Results A total of **80 simulations** were conducted, achieving a **100% success rate** (no failures), with **98.75% of insertions being safe**.

- Unsuccessful: 0%.
- Unsafe Success: 1.25%.
- Safe Success: 98.75%.

Comments

The vast majority of insertions (79 out of 80) were successful and safe, demonstrating the effectiveness of the `acceleration_open_loop` logic under ideal conditions (no noise).

This marks a significant improvement over the results from `acceleration_example`, where only 25% of insertions were safe.

Only 1 out of 80 simulations resulted in a successful but unsafe insertion, suggesting that the logic reliably calculates safe.

There were no insertion failures, indicating that the open-loop strategy is robust in finding insertion opportunities when no noise or uncertainty is present.

These results are based on noise-free simulations, which may not reflect real-world conditions where sensor errors and environmental uncertainties exist.

Performance comparison

Metric	**Example** (`acceleration_example`)	**Open-Loop** (`acceleration_open_loop`)
Total Simulations	80	80
Unsuccessful Insertions	2 (2.5%)	0 (0%)
Unsafe Success	58 (72.5%)	1 (1.25%)
Safe Success	20 (25%)	79 (98.75%)
Success Rate	97.5%	100%
Safe Success Rate	25%	98.75%

Table 1: Performance comparison between the Example Program and Open-Loop Control.

Aspect	**Example** (`acceleration_example`)	**Open-Loop** (`acceleration_open_loop`)
Strengths	High success rate; simple and fast logic.	High safety rate; robust decision-making; dynamic adjustments.
Weaknesses	Poor safety performance; lacks noise handling or safety constraints.	More complex logic may increase computational effort.

Table 2: Strengths and weaknesses of the Example Program and Open-Loop Control.

The open-loop control is better suited for scenarios requiring high safety and robust performance.

The example program can serve as a simple baseline but requires significant modifications to ensure safety and reliability for real-world use.

2 Taking sensor measurement uncertainties into account

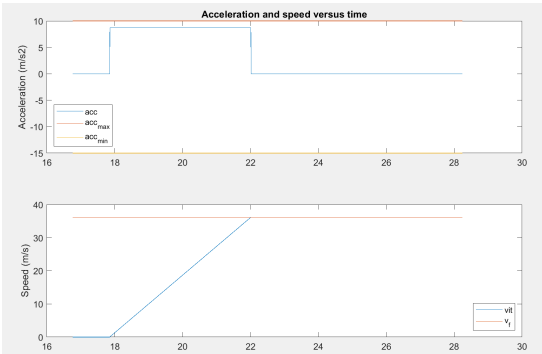
We can test the Acceleration Example by launching the main script $simufile.m$ and setting $question = 2$ in the script.

```
case 2 % question 2
    % The information is now noisy
    % GPS errors are several meters
    for i= 1:N_file
        File(i).pos_est = File(i).pos+Sigma_pos_GPS*(2*rand-1);
    end
    vf_est = vf + Sigma_vit_GPS*(2*rand-1);
    acceleration_open_loop; % Script execution
```

Figure 11: question=2

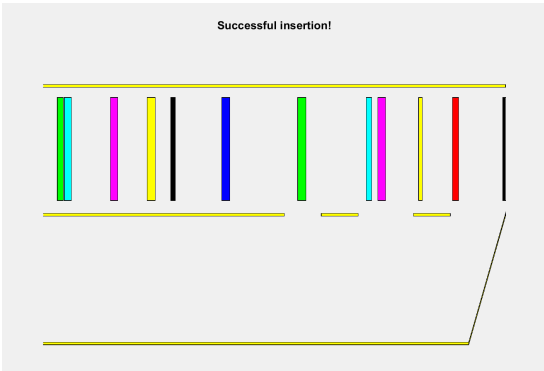
Following are the Simulation Window and the velocity and acceleration curves:

(a) Simulation Window

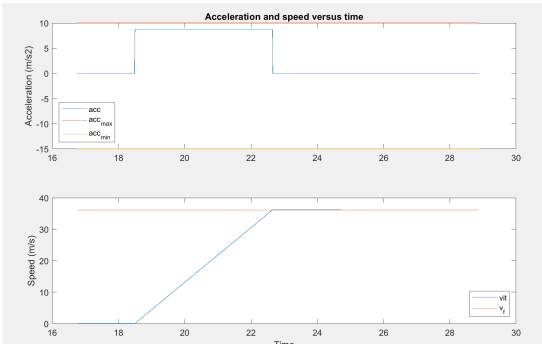


(b) Acceleration and speed versus time

Figure 12: Open loop with sensor measurement uncertainties - Unsafe insertion

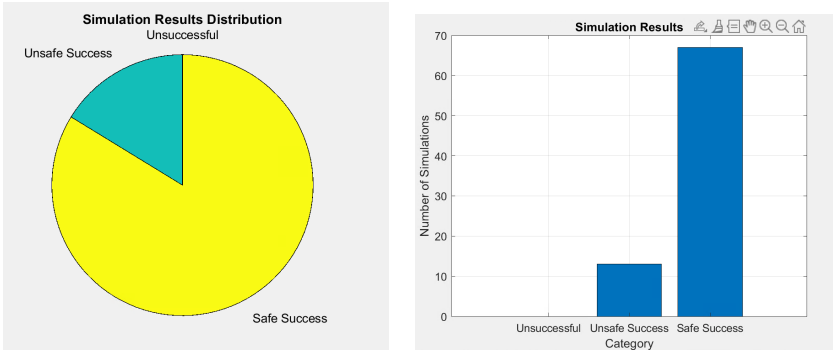


(a) Simulation Window



(b) Acceleration and speed versus time

Figure 13: Open loop with sensor measurement uncertainties - Safe insertion



(a) Open loop with sensor measurement uncertainties (pie-chart)

(b) Open loop with sensor measurement uncertainties (bar-chart)

Figure 14: Open loop with sensor measurement uncertainties - Analysis of Diary

```
Unsuccessful insertions: 0
Successful but unsafe insertions: 13
Successful and safe insertions: 67
```

Figure 15: Result of open loop with sensor measurement uncertainties

A total of 80 simulations were conducted, achieving a 100% success rate for insertions, with 83.75% of the insertions classified as safe.

Introducing sensor measurement noise (e.g., GPS uncertainties) slightly reduced the safe success rate from 98.75% (in the noise-free case, question = 1) to 83.75%.

The noise resulted in 13 unsafe insertions (16.25%), indicating a loss of precision in insertion decisions due to imperfect vehicle position and velocity estimates.

3 Closed-loop control

3.1 Closed-loop feedback equations

The acceleration is then calculated by a proportional-derivative control law with respect to the vehicle being followed:

$$a(t) = \lambda\left(\dot{x}_i(t) - \dot{x}(t)\right) + \mu\left(x_i(t) - L_i - x(t)\right) \tag{1}$$

Here, λ and μ are the derivative and proportional gains respectively. L_i is the position offset relative to the vehicle being followed.

For a closed-loop feedback system, we have the error which is given by:

$$e(t) = x_i(t) - x(t).$$

Taking the derivative:

$$\dot{e}(t) = \dot{x}_i(t) - \dot{x}(t) = v_f - \dot{x}(t),$$

and the second derivative:

$$\ddot{e}(t) = -\ddot{x}(t).$$

Since $\dot{x}(t) = a(t)$:

$$a(t) = -\ddot{e}(t).$$

Thus, the original equation becomes:

$$-\ddot{e}(t) = \lambda\dot{e}(t) + \mu\left(e(t) - L_i\right),$$

which leads us to the transfer function by taking the Laplace transform:

$$\frac{e(s)}{L_i} = \frac{\mu}{s^2 + \lambda s + \mu}.$$

3.2 Calculation of closed-loop parameters

By comparing the transfer function for the closed-loop system with the canonical form of a second-order transfer function:

$$\frac{C(s)}{R(s)} = \frac{\omega_n^2}{s^2 + 2z\omega_n s + \omega_n^2},$$

we can compare the variables, which leads to:

$$\omega_0 = \sqrt{\mu},$$

and

$$2z\omega_0 = \lambda \implies z = \frac{\lambda}{2\omega_0}.$$

Also, the system response time is given by:

$$T_{5\%} = \frac{3}{z\omega_0}.$$

Using a damping factor of $z = 1$, we get by substitutions:

$$\mu = \left(\frac{3}{T_{5\%}}\right)^2,$$

and

$$\lambda = 2\omega_0 = \frac{6}{T_{5\%}}.$$

We find the 5% response time by using the fact that we want the insertion to be done before the end of the insertion area:

$$T_{5\%} = \frac{L_0}{4v_f}.$$

3.3 Simulation

3.3.1 Script for the Closed-loop control strategy

The closed-control loop comes into action when the car has reached a steady velocity. We create a new script for the closed-loop acceleration. The following changes were added to the script, using the calculations performed previously:

```
if (t > t0 + Tf)
    disp('We have entered closed loop!!')
    if (abs(pos+L0/2) < vit*deltaT/2)
     disp('We have passed -L0/2. We can insert.');
    end
    T5_percent = L0 / (4 * vf_est); % T5 %
    lambda = 6 / T5_percent; % Derivative gain
    mu = (3 / T5_percent)^2; % Proportional gain
    Dxx = pos - (File(N_suivi+1).pos_est + Dsar + D0); % Safety margin
    Li = D0 + Dxx + Dsav; % Target offset for closed-loop insertion
```

```
    e = File(N_suivi).pos_est - Li - pos; % Position error
    e_dot = vf_est - vit; % Speed error
    acc = lambda * e_dot + mu * e;
end
```

Where D_{xx} was calculated using the following equation:

$$x_a(t) = x_{i+1}(t) + D_{sar} + D_0 + D_{xx}$$

It is to be noted that before the condition $t > t_0 + T_f$ is met, we are still in the open-loop acceleration phase, which means that the acceleration is according to the predefined value which we calculated. The closed-loop acceleration helps the car to adjust its speed once it reaches constant velocity in the control zone, leading to a safer insertion.

3.3.2 Results and Graphs

We can test the closed control-loop acceleration by launching the main script `simufile` and setting `question=3` in the script. Following are some of the velocity and acceleration curves obtained by the new acceleration script:

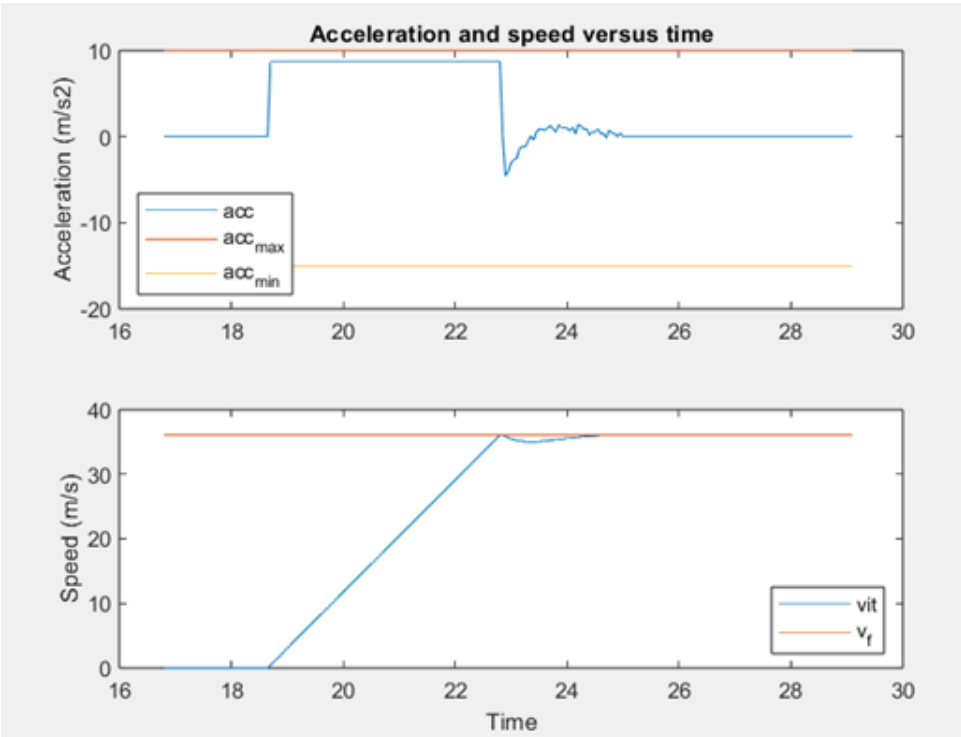


Figure 16: Successful Insertion

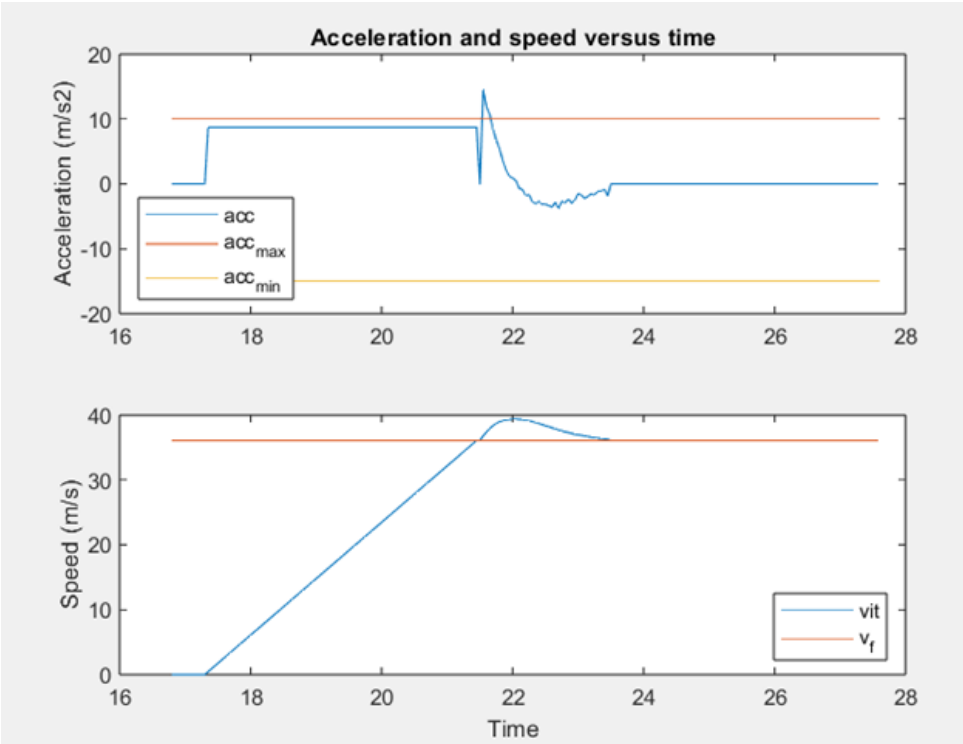


Figure 17: Unsafe Insertion

Case-1 (Successful Insertion):

- The initial acceleration is according to the open-loop acceleration we determined.
- When the vehicle enters the closed-loop phase, the acceleration drops suddenly and increases until the desired velocity is reached. There is a relatively smooth transition for both the acceleration and the velocity.
- The vehicle is inserted safely by respecting the safety distances.

Case-2 (Unsafe Insertion):

- The acceleration graph is initially similar to the open-loop phase of the successful case. However, during the closed-loop phase there is a sharp deceleration and then a significant acceleration above $a_{\max}$. We also observe that there are some oscillations, which indicate instability. Thus, the vehicle slightly struggles to stabilize itself.
- The velocity of the car is increasing constantly. Around 22 seconds, the velocity of the vehicle slightly surpasses the maximum speed limit, indicating a slight overshoot. This indicates that the proportional and derivative gains may need to be calibrated again.
- The vehicle is inserted but does not respect the safety distances.

3.3.3 Impact of Negative D_{xx}:

1. When $D_{xx} < 0$, the merging vehicle does not have enough space to safely insert itself between the two vehicles while respecting safety distances.
2. Negative D_{xx} increases the likelihood of collisions or unsafe interactions during insertion.

Can Safe Insertion Still Be Achieved? While it's challenging, safe insertion might still be possible under specific conditions:

1. **If Localization Errors are Corrected:**
   - Localization errors can cause D_{xx} to appear negative even when there is enough space. If real-time localization updates are available, D_{xx} can be recalculated to reflect accurate values, potentially showing a positive margin.
2. **Reduced Safety Margins:**
   - If the system dynamically adjusts the safety margins (D_0, D_{sr}, D_{sf}), it might allow for a tighter but still safe insertion.
   - This is a trade-off between strict safety compliance and completing the insertion.
3. **Intelligent Maneuvering:**
   - The system could wait for a larger gap by slowing down or speeding up slightly to create a positive D_{xx}.
   - Alternatively, the system could reroute or abort the insertion if no safe option is available.

Recommendation:

- When D_{xx} is negative, the system should:
  - Enter a **waiting mode** until the conditions improve (e.g., larger gap becomes available).
  - Trigger an **emergency stop** or **reroute the vehicle** to avoid collisions.

3.3.4 Evaluation of performance of the closed-loop strategy:

In order to perform several tests, we use the `launch_simufile.m` script with the range of `i` as 1:80. In this way, we have a large enough sample size to study the effect of different strategies on the outcome. It is to be noted that we have slightly modified the `launch_simufile.m` script in order to do a statistical comparison of the data.

Firstly, we run the script for the default open-loop acceleration only. The results are characterized by the following pie-chart and bar-chart:

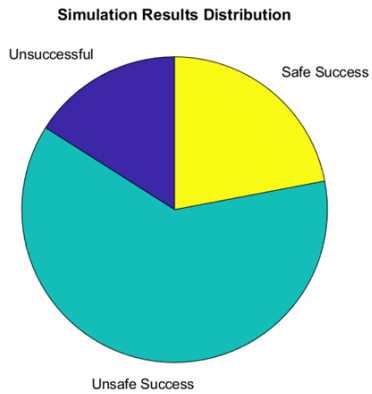


Figure 18: Example Open-loop acceleration only (pie-chart)

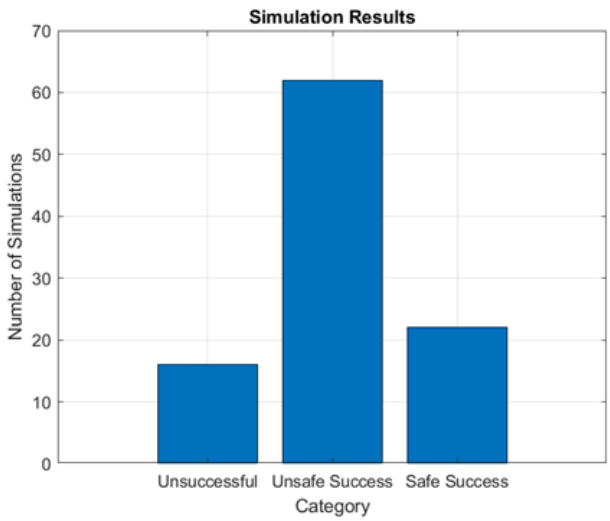


Figure 19: Example Open-loop acceleration only (bar-chart)

Now, we run the script for open-loop plus closed-loop acceleration, by setting `question=3` in the main script `simufile.m`. We obtain the following results:

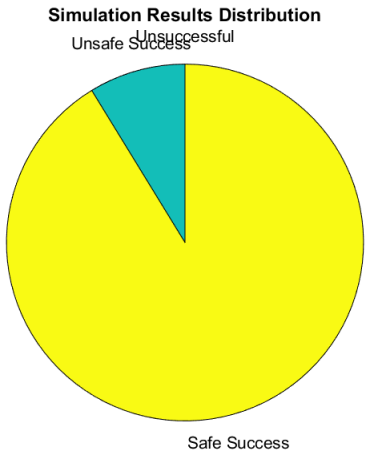


Figure 20: Closed-loop acceleration (pie-chart)

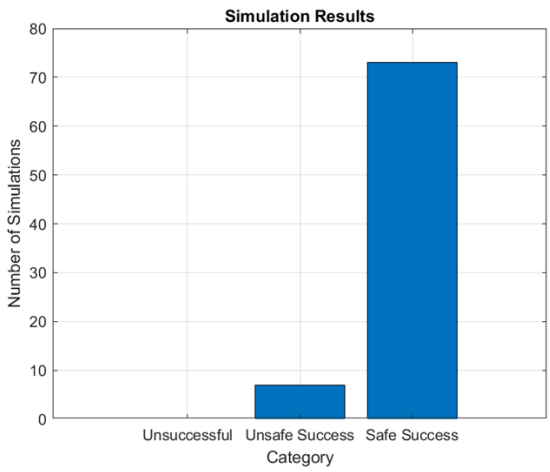


Figure 21: Closed-loop acceleration (bar-chart)

Remarks: The results highlight a clear distinction between the two control strategies. For the open-loop acceleration strategy, the majority of the simulations fall under the *Unsafe Success* category, with relatively fewer instances of *Safe Success* and a significant number of *Unsuccessful* attempts. This indicates that while the open-loop system achieves some degree of insertion, it struggles to ensure safety compliance due to its inability to adapt dynamically to the environment.

In contrast, the combination of open-loop and closed-loop acceleration shows a substantial improvement in performance. The number of *Safe Success* insertions significantly increases, while the *Unsafe Success* category is reduced, demonstrating that the feedback mechanism of the closed-loop phase effectively stabilizes the vehicle and ensures better adherence to safety margins.

This comparison underscores the importance of integrating feedback control for dynamic adjustment of acceleration, which allows the vehicle to adapt to real-time conditions, leading to safer and more reliable insertions. Overall, the open-loop and closed-loop combination outperforms the open-loop-only strategy in terms of both safety and success rates.

4 Additional Work

In this part, we try to find the acceleration limits which give us the best results. First, we clamp the acceleration in the closed loop part to ensure it doesn't exceed the limits. We do this by removing the commented out part in the script as follows:

```
if (t > t0 + Tf)
    if (abs(pos+L0/2) < vit*deltaT/2)
     disp('We have passed -L0/2. We can insert.');
    end
    T5_percent = L0 / (4 * vf_est); % T5 %
    lambda = 6 / T5_percent; % Derivative gain
    mu = (3 / T5_percent)^2; % Proportional gain
    Dxx = pos - (File(N_suivi+1).pos_est + Dsar + D0); % Safety margin
    Li = D0 + Dxx + Dsav; % Target offset for closed-loop insertion
    e = File(N_suivi).pos_est - Li - pos; % Position error
    e_dot = vf_est - vit; % Speed error
    acc = lambda * e_dot + mu * e;
    % Apply saturation
    acc = max(decel_max, min(acc, accel_max)); % Clamp acc within saturation
        limits
```

After running multiple tests with different values for the maximum acceleration (by modifying the parameters.m script), we obtained the best value for the most efficient insertions. This value turned out to be 6 m/s^2. The results obtained are as follows:

```
Unsuccessful insertions: 0
Successful but unsafe insertions: 9
Successful and safe insertions: 71
```

Figure 22: Closed-loop acceleration (bar-chart)

For the default acceleration of 10 m/s^2 we obtained the following results:

```
Unsuccessful insertions: 0
Successful but unsafe insertions: 15
Successful and safe insertions: 65
```

Figure 23: Closed-loop acceleration (bar-chart)

Hence, we can conclude that the optimal saturation value for the maximum acceleration is 6 m/s^2. .